biohub-cell-tracking-during-development

11th Place Solution

Rank	11
Team	tanbo
Author	tanbo
Topic ID	745092
Version	Original

Source: https://www.kaggle.com/competitions/biohub-cell-tracking-during-development/discussion/745092
Retrieved with Kaggle CLI on 2026-10-04.

First of all, I would like to thank the Biohub organizers for sharing the official implementation, and the participants who shared code, papers, and discussions.

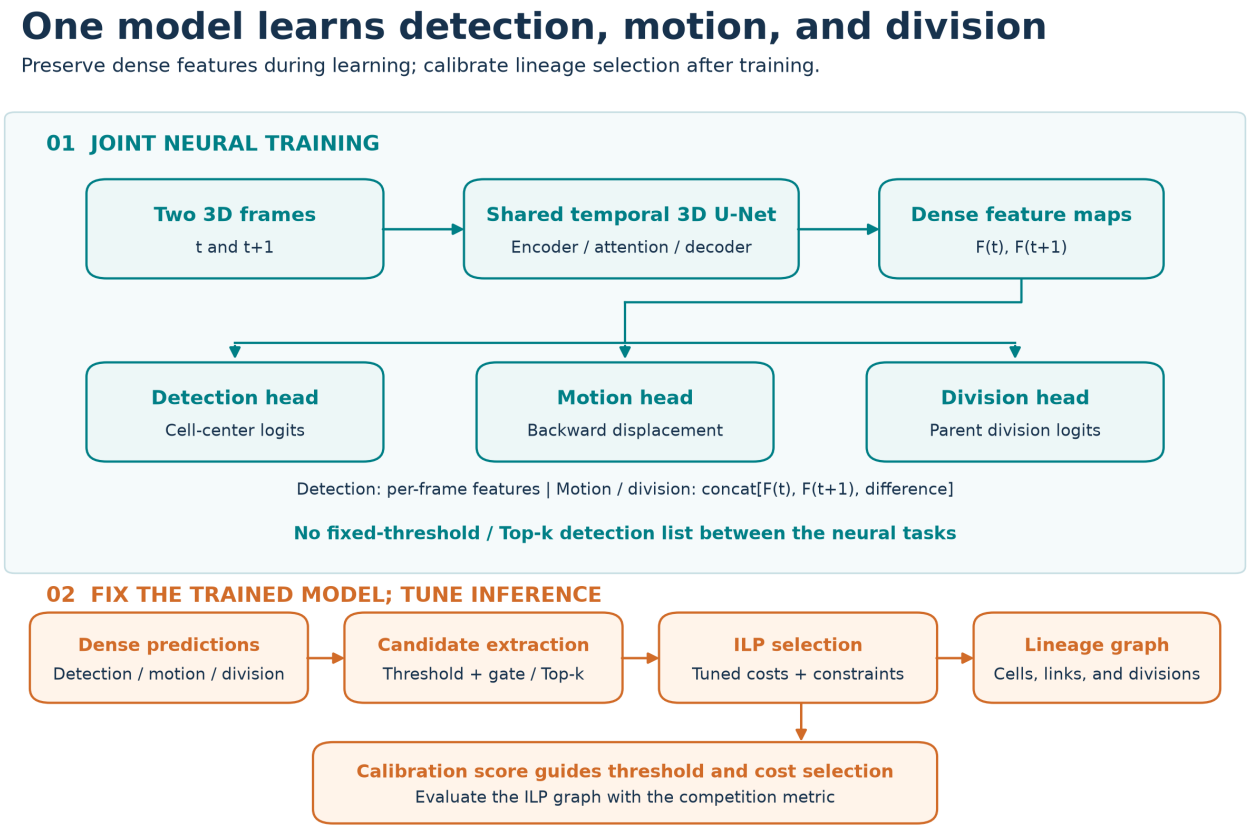
My solution finished **11th and earned a gold medal**, with **0.94905 Public / 0.94743 Private** for the better of my two selected submissions. I focused on transfer to unseen embryos, preserving dense detection features for tracking during training, and calibrating lineage selection after training.

1. Overview

Detection, backward motion, and division prediction were learned together as one model. A shared temporal 3D U-Net supplied features to three heads, whose combined loss updated the network in one training step. Detector pretraining provided the initialization; the backbone and all heads remained trainable during joint learning and real-data fine-tuning.

The pipeline consisted of three learning stages followed by score-based calibration and graph optimization:

1. Train the detector using real annotations and synthetic data.
2. Add motion and division heads, and jointly train the detection, motion, and division components using real and synthetic supervision.
3. Fine-tune on sparsely annotated real data.
4. Fix the trained model and tune detection thresholds, candidate settings, and ILP costs and constraints to maximize the calibration score.
5. Reconstruct lineages with ILP and apply graph and coordinate post-processing.



Gradients flow through the neural model. Candidate extraction and ILP run after training.

Figure 1. Joint neural training followed by calibrated candidate selection and ILP lineage reconstruction. Thresholds and parent-candidate limits are applied after training.

2. Validation: prioritize transfer to another embryo

The labeled training data came from only two embryos, while the hidden test data came from different embryos. I therefore used experiments that trained on one embryo and evaluated on the other when comparing architectures, synthetic data, and augmentation.

I tuned detection thresholds and graph parameters on calibration data after training. With those settings fixed, I then evaluated the resulting pipeline on validation data to assess its final performance.

The final experiment used both embryos. Given the larger dataset and available compute, I considered five-fold cross-validation too costly and used a hold-out split instead: 199 videos were divided into 159 training, 20 calibration, and 20 validation videos, with both embryos represented in each split.

There is an important limitation: some later validation videos had already been involved in earlier detector selection or calibration. Those results were useful for comparing subsequent changes, but they were not a completely untouched estimate of the full pipeline's generalization. Repeatedly comparing changes on the other embryo also makes that embryo part of model development.

3. Detection and temporal prediction

The **two-frame temporal 3D U-Net** predicted cell-center scores. Following Linajea, the motion head estimated where each later-frame cell came from in the earlier frame. The residual between this predicted position and a candidate predecessor provided association evidence.

I deliberately avoided fixed-threshold or Top-k detection selection between the neural tasks during training. The motion and division heads received **dense shared features**, rather than a filtered detection list or the final detection scores. This preserved evidence that would otherwise be discarded before tracking could use it.

Motion and division losses updated the shared U-Net feature extractor; the final detection head received the detection loss. All three tasks remained trainable. Hard peak extraction and calibrated thresholds were applied after training to create graph candidates. ILP was a separate optimization step, with no gradients through the solver.

For sparse real annotations, I downweighted unannotated locations instead of treating them as reliable background. One training configuration used positive/background/unannotated detection weights of 4/1/0.03. Synthetic examples supplied dense supervision from known rendered cells and lineages.

4. Synthetic data and the role of FOCUS-3D

Sparse annotations do not describe every cell in a volume. I used **FOCUS-3D** center estimates from training images to inform synthetic scenes, with statistics from the following sources:

Synthetic property	Source
Cell counts, spatial occupancy, and nearest-neighbor spacing	FOCUS-3D center estimates on training videos

Synthetic property	Source
Movement speed and directional persistence	Annotated training trajectories
Division frequency and daughter separation	Annotated training divisions
Approximate cell appearance, background, and noise	Training-image measurements

FOCUS-3D center estimates were not used as ground-truth real temporal links or division events. They informed spatial statistics; motion and division statistics came from the annotated graphs.

The generator rendered videos with known centers, links, and divisions. The twelve-frame version supplied eleven adjacent two-frame training pairs per video, adding dense lineage supervision alongside sparse real annotations.

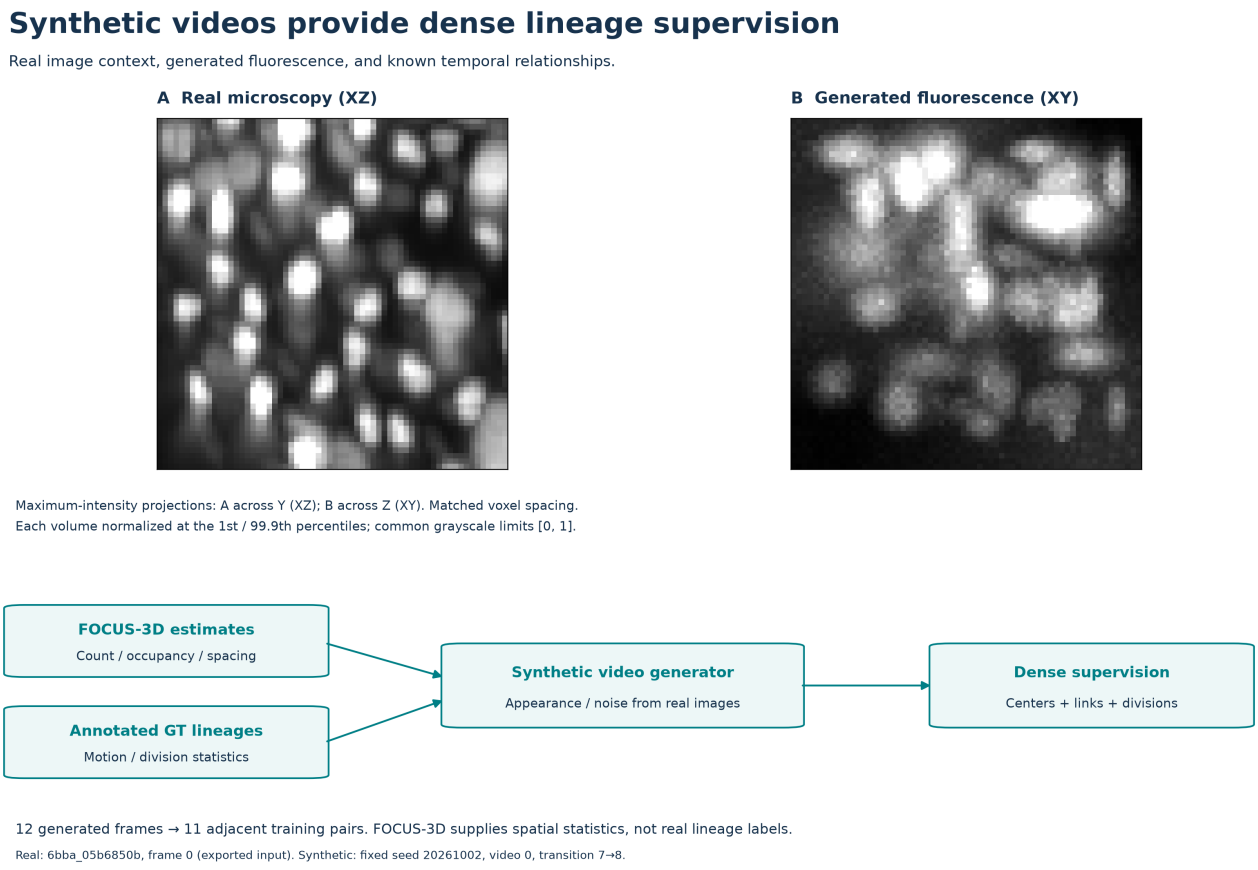


Figure 2. A: real microscopy, XZ maximum-intensity projection across Y. B: synthetic fluorescence, XY projection across Z. The projection directions differ. Both volumes use matched voxel spacing and separate intensity normalization. The synthetic panel was regenerated from saved training statistics for illustration; this is not an accuracy comparison.

I also used a FOCUS-derived affine stream: an image and its affine warp formed a two-frame sample, with correspondences obtained from the transformed center identities. This stream supplied synthetic correspondences, but not positive division labels.

The synthetic renderer approximated real appearance. It was not an exact reconstruction of instance masks, so I judged its usefulness using real-video tracking evaluation, including transfer to another embryo.

5. A learned division prior

The dense division head predicted whether a parent would produce two daughters in the next frame.

For the two frame feature maps, the head used the concatenation of the earlier features, later features, and their difference. The division target was derived from the annotated graph:

- Two outgoing annotated edges: division-positive.
- One outgoing annotated edge: division-negative.
- No outgoing annotated edge: excluded from this loss.

I evaluated the division loss at annotated parent positions. One joint-training configuration used:

```
loss = detection_loss + motion_loss + 2 × division_loss
```

The division-positive BCE weight was 10. These were experimental choices.

The head supplied a **parent-specific division prior**. It did not identify the daughters by itself. Candidate daughter edges and graph constraints were still required to form a division.

6. Real-data fine-tuning

After learning with mixed supervision, I fine-tuned on the sparse real data. This stage kept detection, motion, and division trainable, while removing the rendered synthetic and FOCUS-derived training streams.

One real-only fine-tuning run resumed from joint-training epoch 30, inherited the optimizer state, and started at a learning rate of 1e-4 with cosine decay. Its repair submission used the cumulative epoch-40 checkpoint. After fine-tuning, I fixed the model for inference calibration.

7. After training: tune selection and ILP for the score

I felt that the official implementation's default ILP costs did not produce the best competition score for my model. The solver minimizes a graph cost, while the competition evaluates the resulting lineage with a different metric.

With the trained model fixed, I tuned link, appearance, disappearance, and division costs so that optimizing the ILP objective produced better-scoring lineages on calibration data. I also calibrated detection thresholds and candidate settings.

My score started improving as soon as I began tuning the ILP costs this way. Changing lineage selection without retraining the model was a major turning point.

For example, one repair configuration used the backward-motion residual between a predicted previous position and a candidate predecessor:

```
edge cost = 0.1 × backward-motion residual in µm − 0.25
appearance cost = 0.2
disappearance cost = 0.2
division cost = 0.5 − predicted division probability
```

This configuration used a 13 µm candidate gate and at most five candidate parents. These limits applied to the inference graph, not the neural training interface. A division required two selected daughter edges; the learned parent prior reduced the division cost at likely mothers.

I searched combinations of costs and constraints, solved each ILP, and evaluated its lineage using the competition metric. The metric selected the parameters; it did not replace the ILP objective.

8. Submission results

Stage	Public	Private	Absolute Public–Private gap
Baseline	0.82022	0.79007	0.03015
Completed architecture before real-data fine-tuning	0.89618	0.89609	0.00009
Fine-tuned model with tuned ILP	**0.94905**	**0.94743**	0.00162
Additional post-processing	0.94846	0.94703	0.00143

From the completed architecture to fine-tuning plus ILP tuning, Public improved by **0.05287** and Private by **0.05134**. This measures the two changes together and does not isolate their individual contributions.

I also tuned post-processing with Optuna, but the submitted version scored **0.00059 lower on Public and 0.00040 lower on Private** than the fine-tuned model with tuned ILP. No improvement was observed in this submission comparison.

My interpretation is that emphasizing transfer to another embryo helped keep Public and Private scores close, while calibrating ILP costs helped exploit the model's predictions. The submission history supports the combined improvement, without establishing a separate causal gain for each component.

9. Working with AI coding agents

I used AI coding agents, keeping relevant papers, the official implementation, and discussion copies in the repository for them to consult before implementing changes.

This made implementation decisions easier to inspect. Its score contribution was not measured.

10. What I would improve

Train with both embryos earlier. Transfer experiments were useful, but prolonged one-embryo development delayed broader training and left less time for completed submissions.

Keep an independent final evaluation set. Preserve its isolation from detector selection and calibration throughout development.

Run matched ablations and record their gains. Compare dense joint learning against a discrete-detection training interface, then compare fine-tuning, ILP calibration, and individual post-processing operations under matched conditions. This would distinguish architecture gains from later selection gains and justify added complexity.

References

- Biohub — Cell Tracking During Development
- Official U-Net baseline inference notebook
- Linajea: Automated reconstruction of whole-embryo cell lineages by learning from sparse annotations